L'ÉDITION — Frontend Admin Authentication & Concurrent Refresh Guide

> **Target API:** `MagizineAdmin.Api` (`http://localhost:5024` | `https://localhost:7036`)
>
> **Architecture:** Short-Lived JWT `AccessToken` (`60m`) + Rotating `RefreshToken` (`7d`) + `HttpOnly` Cookies + **Frontend Mutex Queue** + **Backend 30-Second Grace Period**

---

1. End-to-End Authentication & Concurrent Refresh Flow

```
sequenceDiagram
    autonumber
    actor Browser as Frontend (Multi-Tabs / Parallel Requests)
    participant API as MagizineAdmin.Api
    participant Svc as AuthService
    participant DB as SQL Server (Tbl_Admin)

    Note over Browser,DB: 1. Initial Login (POST /api/admin/auth/Login)
    Browser->>API: POST /api/admin/auth/Login { UsernameOrEmail, Password }
    API->>Svc: LoginAsync(req)
    Svc->>DB: Verify BCrypt PasswordHash & Save RefreshTokenHash (SHA-256)
    API-->>Browser: Set-Cookie: admin_access_token (HttpOnly, Path=/)\nSet-Cookie: admin_refresh_token (HttpO

    Note over Browser: 2. Requests A, B, and C fail simultaneously with 401
    Browser->>Browser: Acquire Mutex lock (isRefreshing = true); push Requests B & C into failedQueue
    Browser->>API: Request A calls POST /api/admin/auth/Refresh (Sends Token 1 Cookie)
    API->>Svc: RefreshTokenAsync(Token 1)
    Svc->>DB: Verify SHA256(Token 1) == RefreshTokenHash
    Svc->>DB: Move Token 1 -> PreviousRefreshTokenHash (Grace Period: UtcNow + 30s)
    Svc->>DB: Save SHA256(Token 2) -> RefreshTokenHash (Expires: UtcNow + 7d)
    API-->>Browser: Set-Cookie (AccessToken + Token 2) + 200 OK
    Browser->>Browser: Release Mutex lock; retry queued Requests B & C

    rect rgb(255, 240, 240)
    Note over Browser,DB: 3. Fallback: Another browser tab triggers Refresh with Token 1 within 30s
    Browser->>API: Tab 2 calls POST /api/admin/auth/Refresh (Still carrying Token 1)
    API->>Svc: RefreshTokenAsync(Token 1)
    Svc->>DB: Verify SHA256(Token 1) == PreviousRefreshTokenHash & PreviousRefreshTokenExpiresAt > UtcNow
    Note over Svc,DB: Grace Period Hit: Issue AccessToken without rotating RefreshTokenHash again
    API-->>Browser: Set-Cookie (AccessToken, preserving Token 2 in Cookie Jar) + 200 OK (Session preserved)
    end
```

2. Cookie & Token Specification

Cookie / Token	Name	Format	Lifetime	Flags	Path Scope
Access Token Cookie	`admin_access_token`	Signed HMAC-SHA256 JWT (`typ: "access"`)	`60 minutes`	`HttpOnly` , `Secure` , `SameSite=Lax`	`/` (Sent on all Admin API calls)
Refresh Token Cookie	`admin_refresh_token`	64-byte CSPRNG (`Base64Url`)	`7 days`	`HttpOnly` , `Secure` , `SameSite=Strict`	`/api/admin/auth` (Sent only to `/Refresh` & `/Logout`)
Grace Period Record	`PreviousRefreshTokenHash`	SHA-256 hex digest in `dbo.Tbl_Admin`	`30 seconds`	Server-side DB column	Prevents multi-tab race conditions

3. Admin Auth Endpoints Reference (`/api/admin/auth`)

3.1 `POST /api/admin/auth/Login` (`[AllowAnonymous]`)

Authenticates an Admin by `Username` (`superadmin`) or `Email` (`admin@ledition.com`).

Request Body:

```
{
  "UsernameOrEmail": "superadmin",
  "Password": "Admin@123"
}
```

Response (`200 OK`):

- Sets `HttpOnly` cookies `admin_access_token` and `admin_refresh_token` .

```
{
  "IsSuccess": true,
  "IsError": false,
  "RespType": "Success",
  "RespCode": "MS#000",
  "RespDesp": "Signed in successfully.",
  "Data": {
    "AccessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "TokenType": "Bearer",
    "ExpiresAt": "2026-10-11T11:30:00+00:00",
    "RefreshTokenExpiresAt": "2026-10-18T10:30:00+00:00",
    "SessionId": "01J...",
    "IsGracePeriodHit": false,
    "Admin": {
      "AdminId": "01JADMIN00000000000000001",
      "Username": "superadmin",
      "Email": "admin@ledition.com",
      "Role": "SuperAdmin",
      "IsActive": true,
      "LastLoginAt": "2026-10-11T10:30:00+00:00",
      "CreatedAt": "2026-10-10T00:00:00+00:00",
      "UpdatedAt": "2026-10-11T10:30:00+00:00"
    }
  }
}
```

3.2 `POST /api/admin/auth/Refresh` (`[AllowAnonymous]`)

Rotates the session using the `admin_refresh_token` `HttpOnly` cookie (or optional `{"RefreshToken": "..."}` fallback body for non-browser clients).

- **Standard Rotation (`IsGracePeriodHit: false`)**: Moves current `RefreshTokenHash` to `PreviousRefreshTokenHash` (`UtcNow + 30s`), saves new `RefreshTokenHash`, and sets both `admin_access_token` and `admin_refresh_token` `HttpOnly` cookies.
- **30s Grace Period Hit (`IsGracePeriodHit: true`)**: Triggered if a second browser tab sends the just-rotated token within 30 seconds. Mints a fresh `AccessToken` and updates `admin_access_token` without invalidating the active refresh token.

Request Body:

```
{}
```

3.3 `GET /api/admin/auth/Me` (or `POST /api/admin/auth/Me`) (`[Authorize]`)

Used on Frontend app boot/reload to verify the `HttpOnly` cookie session and hydrate the logged-in Admin state.

Response (`200 OK`):

```
{
  "IsSuccess": true,
  "IsError": false,
  "RespType": "Success",
  "RespCode": "MS#000",
  "RespDesp": null,
  "Data": {
    "AdminId": "01JADMIN00000000000000001",
    "Username": "superadmin",
    "Email": "admin@ledition.com",
    "Role": "SuperAdmin",
    "IsActive": true,
    "LastLoginAt": "2026-10-11T10:30:00+00:00",
    "CreatedAt": "2026-10-10T00:00:00+00:00",
    "UpdatedAt": "2026-10-11T10:30:00+00:00"
  }
}
```

3.4 `POST /api/admin/auth/UpdateProfile` (`[Authorize]`)

Updates the current Admin's `Username` and `Email`.

```
{
  "Username": "superadmin",
  "Email": "admin@ledition.com"
}
```

3.5 `POST /api/admin/auth/ChangePassword` (`[Authorize]`)

Verifies `CurrentPassword`, hashes `NewPassword`, revokes both `RefreshTokenHash` and `PreviousRefreshTokenHash`, and clears the `HttpOnly` cookies so the user signs in cleanly with the new password.

```
{
  "CurrentPassword": "Admin@123",
  "NewPassword": "Admin@1234",
  "ConfirmNewPassword": "Admin@1234"
}
```

3.6 `POST /api/admin/auth/Logout` (`[AllowAnonymous]`)

Nullifies `RefreshTokenHash`, `RefreshTokenExpiresAt`, `PreviousRefreshTokenHash`, and `PreviousRefreshTokenExpiresAt` in `dbo.Tbl_Admin` and deletes both `HttpOnly` cookies.

```
{}
```

---

4. Production Frontend Mutex Queue Interceptor (Axios / TypeScript)

Use this copy-pasteable Axios client in the Admin Frontend SPA. It automatically sends `HttpOnly` cookies (`withCredentials: true`) and implements the **`isRefreshing` + `failedQueue` Mutex** so concurrent `401` responses trigger only **one** `/api/admin/auth/Refresh` request:

```ts
import axios, { AxiosError, InternalAxiosRequestConfig } from 'axios';

interface RetryableRequestConfig extends InternalAxiosRequestConfig {
  _retry?: boolean;
}

interface QueuedRequest {
  resolve: () => void;
  reject: (error: unknown) => void;
}

export const adminApi = axios.create({
  baseURL: import.meta.env.VITE_ADMIN_API_URL ?? 'http://localhost:5024',
  withCredentials: true, // CRITICAL: Sends & receives HttpOnly cookies automatically
  headers: {
    'Content-Type': 'application/json',
  },
});

let isRefreshing = false;
let failedQueue: QueuedRequest[] = [];

const processQueue = (error: unknown = null) => {
  failedQueue.forEach((prom) => {
    if (error) {
      prom.reject(error);
    } else {
      prom.resolve();
    }
  });
  failedQueue = [];
};

adminApi.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as RetryableRequestConfig | undefined;

    if (!originalRequest || error.response?.status !== 401) {
      return Promise.reject(error);
    }

    // Never loop on Login or Refresh endpoints themselves
    if (
      originalRequest.url?.includes('/api/admin/auth/Login') ||
      originalRequest.url?.includes('/api/admin/auth/Refresh')
    ) {
      return Promise.reject(error);
    }

    if (originalRequest._retry) {
      return Promise.reject(error);
    }

    // If a refresh is already in flight, queue this request until it finishes
    if (isRefreshing) {
      return new Promise<void>((resolve, reject) => {
        failedQueue.push({ resolve, reject });
      }).then(() => {
        originalRequest._retry = true;
        return adminApi(originalRequest);
      });
```

```
    }

    originalRequest._retry = true;
    isRefreshing = true;

    try {
      const refreshRes = await adminApi.post('/api/admin/auth/Refresh', {});

      if (!refreshRes.data?.IsSuccess) {
        throw new Error(refreshRes.data?.RespDesp || 'Session expired');
      }

      // Wake up and retry all queued 401 requests
      processQueue(null);
      return adminApi(originalRequest);
    } catch (refreshError) {
      processQueue(refreshError);
      window.location.href = '/admin/login';
      return Promise.reject(refreshError);
    } finally {
      isRefreshing = false;
    }
  }
);
```

5. Verification Matrix

Scenario	Test Action	Expected Result
1. Single Request Expiry	Trigger an API call with an expired `AccessToken`	`/Refresh` succeeds and the original call completes normally.
2. Frontend Mutex (Parallel 401s)	Trigger 5 concurrent API requests on dashboard load with an expired token	Exactly **one** `/Refresh` request fires in the network tab; all 5 requests retry and succeed.
3. Backend Grace Period (Multi-Tabs)	Fire `/Refresh` simultaneously from two tabs using the same initial refresh token	Both requests return `200 OK`; second tab hits `PreviousRefreshTokenHash` (`IsGracePeriodHit: true`) and neither tab gets logged out.
4. Expired Grace Period Replay	Replay an old refresh token after the 30-second window expires	Backend rejects the request with `ME#401`.
5. Session Logout	Invoke `POST /api/admin/auth/Logout`	Both `RefreshTokenHash` and `PreviousRefreshTokenHash` are set to `NULL` in `dbo.Tbl_Admin`, and `HttpOnly` cookies are deleted.